

注：

1. 课程内容根据重要程度分为 1~5 五个星级。每一个小节有小节对应的星级；对于小节里面的知识点，如果跟小节的星级不匹配，会单独在知识点的后面添加星级以示区别。

2. 课堂上没有讲到的补充性内容，以蓝色字体标出。

第五层 运输层

重要内容 ★★★★★

无连接的 UDP 的特点。

面向连接的 TCP 的特点。

可靠传输的工作原理。

TCP 的滑动窗口、流量控制、拥塞控制和连接管理。

5.1 运输层协议概述 ★★★

运输层用来解决进程之间的通信问题，向高层屏蔽了下面网络核心的细节。

运输层的两个主要协议： 用户数据报协议 UDP——无连接；

传输控制协议 TCP——面向连接。

运输层的端口

端口是一种逻辑接口，是应用层实体与运输层实体进行层间交互的服务访问点。

接收方的运输层通过端口将应用数据上交给不同的应用进程。

运输层用 16 位的端口号来标志一个端口。

端口号的分类

服务器使用的端口号：0~1023 为熟知端口号，1024~49151 为登记端口号。

客户端使用的端口号：49152~65535。这类端口号仅在客户进程运行时动态分配，由客户进程临时使用。客户进程结束后，其所使用的端口号会被系统收回，以供其他客户进程使用。

常用的熟知端口号

应用程序	FTP	TELNET	SMTP	DNS	TFTP	HTTP	SNMP	SNMP (trap)	HTTPS
熟知端口号	21	23	25	53	69	80	161	162	443

（记住 FTP、TELNET、SMTP、DNS、HTTP、HTTPS 使用的运输层端口号即可）

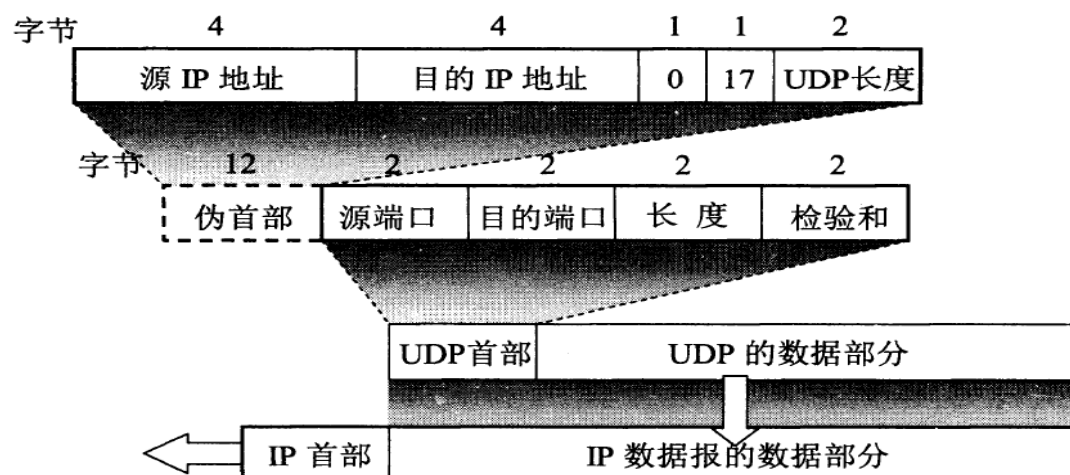
5.2 用户数据报协议 UDP ★★★★★

UDP 概述

特点：

- 无连接；
- 尽最大努力交付；
- 面向报文；
- 没有拥塞控制；
- 支持一对一、多对多、多对一、多对多的交互通信；
- 首部开销小。

UDP 的首部格式



UDP 用户数据报的首部和伪首部

UDP 用户数据报的首部只有 8 个字节,由 4 个字段组成,每个字段都是两个字节。其中长度字段指的是整个 UDP 用户数据报的长度。

在计算检验和时,临时把“伪首部”和 UDP 用户数据报连接在一起计算。伪首部仅仅是为了计算检验和。检验和字段检验的内容为:伪首部、首部和数据部分。

5.3 传输控制协议 TCP 概述 ★★★★★

特点:

- 面向连接;
- 连接类型是点对点(即一对一)连接;
- 可靠交付;
- 提供全双工通信;
- 面向字节流;

TCP 的面向连接

每一条 TCP 连接有两个端点:套接字。套接字 = (IP 地址:端口号)。只要使用不同的端口号,同一个 IP 地址就可以有多个不同的 TCP 连接。

5.4 可靠传输的工作原理 ★★★★★

TCP 在不可靠的网络层之上提供可靠传输的服务,必须采取相关的措施。

1. 停止等待协议:每发送完一个分组就停止发送,等待对方的确认。在收到确认后,再发送下一个分组。

2. ARQ(自动重传请求)协议:为每一个已发送的分组设置一个超时计时器。发送方只要在超时计时器到期之前收到了相应的确认,就撤销该超时计时器,继续发送下一个分组;否则就重传该分组。

分组发送之后必须暂时保留该分组的副本,以备超时重传;每个分组和确认都要进行编号;接收方收到有差错的分组后直接丢弃,等待发送方超时重传。

为了提高发送效率,又提出了以下协议:

3. 连续 ARQ 协议:发送方维持一个发送窗口,位于发送窗口内的分组都可被连续发送出去,而不需要等待对方的确认。接收方一般采用累积确认的方式,即接收方不必对收到的分组逐个发送确认,而是在收到几个分组后,对按序到达的最后一个分组发送确认。这就表示,到这个分组为止的所有分组都已正确收到了。

4. 滑动窗口协议:发送方每收到一个按序到达的确认,就把发送窗口向前滑动一个分组的位置。

连续 ARQ 协议采用 Go-back-N（回退 N）协议进行超时重传。Go-back-N 表示需要再退回来重传已发送过的 N 个分组。

5. GBN 协议：接收方收到了失序的分组，只能对按序到达的最后一个分组发送确认。发送方收到确认后，等待超时会重传曾经发送过的但是未被确认的 n 个分组。

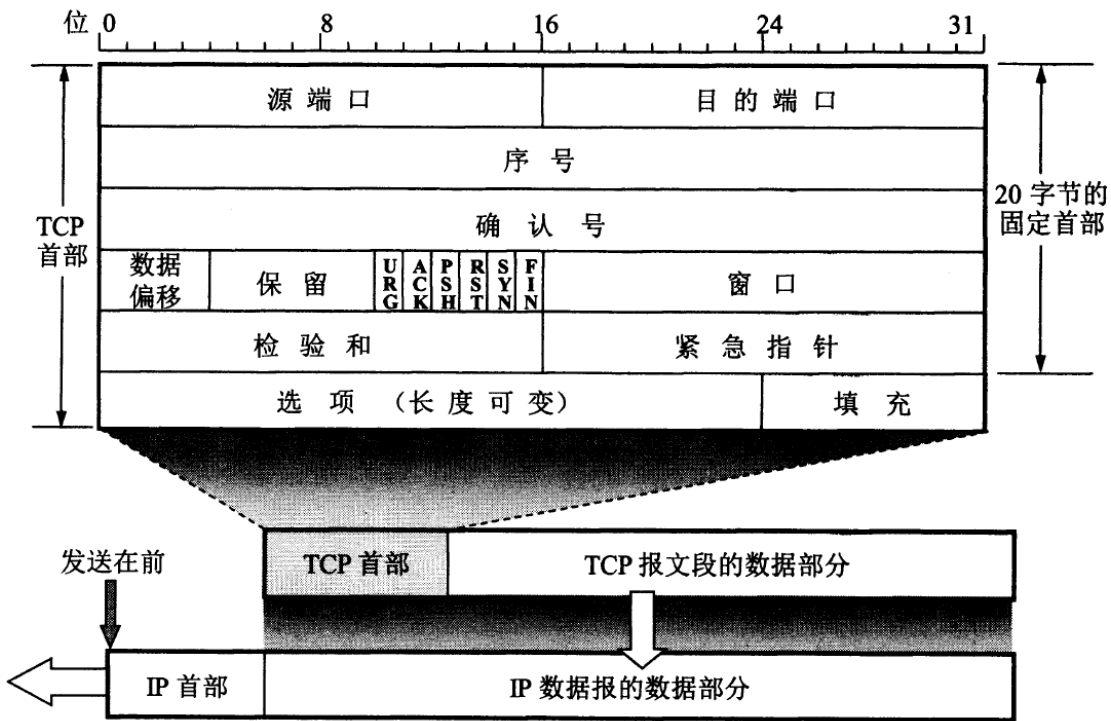
例如：假设发送方发送了 5 个分组，而中间的第 3 个分组丢失了。这时接收方只能对前两个分组发出确认，而发送方只能超时重传后面的三个分组。

GBN 协议的特点：★★★★

- 1) 若采用 n 个比特对分组编号，那么发送窗口的大小 W 应满足： $1 \leq W \leq 2^n - 1$ ；
- 2) 接收窗口大小为 1，所以只能接收期望收到的分组，对于收到的其他分组直接丢弃；
- 3) 如果出现超时，发送方会重传所有已发送但未被确认的分组。

5.5 TCP 报文段的首部格式 ★★★★★

TCP 传送的数据单元是报文段。TCP 报文段的首部格式如下：



TCP 报文段的首部格式

序号 (seq)：占 4 字节。TCP 连接中传送的字节流中的每一个字节都按顺序编号，序号字段的值指的是本报文段所发送的数据的第一个字节的序号。整个字节流的起始序号在连接建立时设置。

确认号 (ack)：占 4 字节，是期望收到对方的下一个报文段的数据的第一个字节的序号。若确认号 = N，则表明：到序号 N-1 为止的所有数据都已正确收到。

确认标志位 ACK：只有当 ACK = 1 时确认号字段才有效；当 ACK = 0 时，确认号无效。除三报文握手的第一个报文段 ACK 为 0，其余所有的报文段 ACK 都为 1。

同步标志位 SYN：在连接建立时用来同步序号。SYN = 1 表示这是一个连接请求或连接接受报文段。此标志位只在三报文握手的前两个报文中有效。

- 当 SYN = 1, ACK = 0 时，表明这是连接请求报文段。
- 当 SYN = 1, ACK = 1 时，表明这是连接接受报文段。

终止标志位 FIN：用来释放一个连接。FIN = 1 表明此报文段的发送方的数据已发送完毕，并要求释放此连接。此标志位只在连接释放报文段中有效。

窗口：占 2 字节，它是发送本报文段的一方的接收窗口。窗口值告诉对方，从本报文段首部中的确认号算起，自己还可以接收（对方还可以发送）多少字节的数据。窗口值是接收方让发送方设置其发送窗口的依据。窗口值是不断动态变化的，以此实现流量控制的功能。

检验和：检验和字段检验的范围包括首部和数据这两部分。在计算检验和时，要在 TCP 报文段的前面加上 12 字节的伪首部。所以检验和检验的内容为：伪首部、首部和数据部分。

选项部分：★★★★

最大报文段长度 MSS：每一个 TCP 报文段中的数据部分的最大长度。

窗口扩大选项：占 3 字节，其中有一个字节表示移位值 S。实际窗口值等于 TCP 首部中的窗口值扩大 2^S 倍。

选择确认：接收方告诉发送方哪些字节流没有按序到达，这样重传时就可以只重传缺少的数据而不重传已经正确到达的数据。选择确认用 4 个字节来指明一个未按序到达的字节块的边界，一个字节块的左、右两个边界需要 8 个字节才能确定。

5.6 TCP 可靠传输的实现 ★★★★★

以字节为单位的滑动窗口

TCP 的滑动窗口是以字节为单位的。TCP 报文段的交互以及窗口的滑动，详见 PPT 或课本相关内容。

5.7 TCP 的流量控制 ★★★★★

流量控制就是接收方动态调整自己的接收窗口的大小（即动态调整窗口字段的值），让发送方的发送窗口进行同样的调整，发送方的发送速率就会降低，接收方就来得及接收和处理收到的数据。流量控制就是利用滑动窗口机制在 TCP 连接上实现接收方对发送方的流量调整。

流量控制的示例详见 PPT 或课本相关内容。

5.8 TCP 的拥塞控制 ★★★★★

在某段时间内，若对网络中某资源（带宽、缓存、处理机等）的需求超过了该资源所能提供的可用部分，网络的性能就要变坏，这种情况称为拥塞。

拥塞控制和流量控制的区别：

拥塞控制就是防止过多的数据注入到网络中，以防网络中的路由器或链路过载。它是一个全局性的过程，涉及到所有的主机和路由器。

流量控制是指在给定的发送端和接收端之间的点对点通信量的控制，即接收端抑制发送端发送数据的速率，以便使接收端来得及接收。

TCP 的拥塞控制方法

- 慢开始：由小到大逐渐增大拥塞窗口的数值，每经过一个轮次，cwnd 就加倍。
- 拥塞避免：拥塞窗口 cwnd 缓慢地增大，每经过一个轮次，拥塞窗口 cwnd 值加 1。
- 慢开始门限 ssthresh：当 $cwnd < ssthresh$ 时，使用慢开始算法；当 $cwnd > ssthresh$ 时，改为执行拥塞避免算法；当 $cwnd = ssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法。
- 当网络出现拥塞时：无论在慢开始阶段还是在拥塞避免阶段，只要发送方判断网络出现拥塞（如超时计时器到期），就要把 ssthresh 设置为出现拥塞时的拥塞窗口值的一半，然后把拥塞窗口 cwnd 重新设置为 1，并执行慢开始算法。
- 快重传：当发送方连续收到三个重复的确认时，就执行“快重传”，这样就不会出现超时，发送方也就不会误认为网络出现了拥塞。
- 快恢复：发送方在快重传丢失的报文段的同时，执行快恢复算法：令拥塞窗口 cwnd 减半，并设置慢开始门限 ssthresh 为同样的数值，然后开始执行拥塞避免算法，使拥塞窗口缓慢地线性增大。

5.9 TCP 的运输连接管理 ★★★★★

三报文握手建立 TCP 连接：

- 1) 客户向服务器发送连接请求报文段：SYN = 1，并选择初始序号 $\text{seq} = x$ 。此报文段不能携带任何数据，但要消耗掉一个序号。
- 2) 服务器如同意建立连接，则向客户发送确认报文段：SYN = 1，ACK = 1，确认号 $\text{ack} = x + 1$ （序号 x 被消耗掉了），同时选择初始序号 $\text{seq} = y$ 。这个报文段也不能携带数据，也同样要消耗掉一个序号。
- 3) 客户收到确认报文段后，还要向服务器给出确认：ACK = 1，确认号 $\text{ack} = y + 1$ （序号 y 被消耗掉了），而自己的序号 $\text{seq} = x + 1$ 。此报文段可以携带数据，但如果不携带数据则不消耗序号。服务器收到客户的确认后，TCP 连接就建立成功了。

四报文挥手释放 TCP 连接：

- 1) 数据传输结束后，客户向服务器发送连接释放报文段：FIN = 1，序号 $\text{seq} = u$ （已传送的最后一个字节的序号加 1），等待服务器的确认。TCP 规定，连接释放报文段即使不携带数据，也要消耗掉一个序号。
- 2) 服务器收到连接释放报文段后发出确认：确认号 $\text{ack} = u + 1$ （序号 u 被消耗掉了），而这个报文段自己的序号 $\text{seq} = v$ （已传送的最后一个字节的序号加 1）。此时从客户到服务器这个方向的连接就释放了，TCP 连接处于半关闭状态。
- 3) 服务器发出连接释放报文段：FIN = 1，序号 $\text{seq} = w$ （在半关闭状态服务器仍然可以向客户发送数据），确认号 $\text{ack} = u + 1$ ，等待客户的确认。
- 4) 客户收到连接释放报文段后，必须给出确认：确认号 $\text{ack} = w + 1$ （序号 w 被消耗掉了），序号 $\text{seq} = u + 1$ 。